SOUTH LAKE HOME EDUCATORS



Lesson 9

Core Mechanics III: On the Scoreboard

Semester 1, Week 9 - Score Tracking + Real On-Screen UI

At a Glance

Lesson 9 objectives & materials

By the end of today, students will:

- Add score tracking to the existing hit-detection code
- Build a persistent, always-visible HUD
- Explain push-based vs. poll-based display updates
- Recognize which engines' text tools were already the real UI

What We're Using Today

- Demo laptop with all 8 engines verified
- GameMaker (GML script), for the live lecture demo - callback to Lesson 5
- No new assets this week - logic and UI lesson
- Unity, Unreal, 001GC: genuinely new UI setup this week
- Your own [YourName]Game project from Lesson 8 - not a new one

IGDA framework tie-in: Programming - UI/UX implementation, state display, event-driven vs. polling architecture (intermediate).

Today's Agenda



0:00 - 0:05	Welcome back, Lesson 8 recap
0:05 - 0:15	The Payoff of Deferring UI
0:15 - 0:35	Score: Reusing the Hit-Detection Path
0:35 - 0:50	Two Ways to Stay in Sync: Push vs. Poll
0:50 - 0:55	Introduce lab exercise, hand out engine-specific handouts
LAB TIME	***— On the Scoreboard —***
0:55 - 1:50	Add score tracking and a real on-screen UI
1:50 - 2:00	Exit check / quick share-out



Welcome Back

Lesson 8 recap - your game got real stakes.

5 min

Welcome Back

5 min

Quick recap of Lesson 8:

- A lives system - an escaped enemy costs a life, tracked starting at 3
- A real lose condition - "Game Over" appears at zero lives
- Today: score and lives finally show on screen the whole time you're playing

Open your own project from last week today, not a new one - and no new assets needed either, this is a logic and UI lesson.



The Payoff of Deferring UI

A promise made in Lesson 6, kept today.

10 min

Keeping a Promise

10 min

Lesson 6: "a plain line of text is completely legitimate today - the real UI system comes later, once there's a score and lives to actually display." That's today.

For some engines, keeping that promise is almost free - the tool was already right, it just needs to run continuously. For a couple, today is genuinely new.

Discussion: why wait until now to build a real UI, instead of doing it back in Lesson 6? (There was nothing worth displaying continuously yet - build the data first, display it once it means something.)



Score: Reusing the Path

The hit-detection code already runs once per kill.

20 min

Score Joins the Kill

20 min

Every engine's hit-detection code already runs exactly once per confirmed kill - the same place enemies_remaining already changes. Score just joins it: +10 per kill.

The math is the easy part. GameMaker demo: obj_bullet's Collision event gets one new line - obj_gamemanager.score += 10 - right alongside the enemies_remaining change already there.

Showing it is the real work this week - a new kind of separation: game logic vs. display.

Display lives in obj_gamemanager's Draw GUI event - the same event already drawing "You Win!" and "Game Over" since Lesson 6.



Push vs. Poll

Two valid ways to keep a display in sync.

15 min

Two Ways to Stay in Sync

15 min

PUSH

Every time the data changes, call an update function to reflect it. Godot, GDevelop, Unreal, 001GC's real-time Field binding.

POLL

Just re-check the data every single frame, no matter what. GameMaker's Draw GUI, TIC-80's print(), and this week's Unity HUD script.

Neither is "more correct." Push means remembering to call the update everywhere data changes. Poll means never forgetting, at the cost of a little wasted work every frame.



Discussion: can you think of something in a game where checking every single frame would start to feel wasteful, if there were hundreds of things to check instead of two?



Today's Exercise & Lab

On the Scoreboard - score tracking and a real HUD.

5 min

On the Scoreboard

5 min

Destroying an enemy adds 10 points. Score and lives are both visible on screen, all the time - built on top of your existing project.

Level 1 - Minimal

Score +10 per kill; score and lives both always visible on screen.

MVP - a complete finish for today.

Level 2 - Complete

Readouts never show a stale or wrong number - always match reality.

Everything works end to end.

Level 3 - Refined

Style the HUD, add scoring variety, tighten logic/display separation.

Ready to share.



Unity, Unreal, 001 Game Creator: today's setup is more involved than usual - genuinely new UI tools. Everyone else: your engine already had the right tool since Lesson 6.

Lab Time Notes



- Unity, Unreal, and 001 Game Creator clusters need the most attention this week - budget extra circulation time, expect setup questions before logic questions.
- 001 Game Creator: forgetting to tick "Update Real-time" on a Field looks like a mystery bug, not an obviously missing setting - check this first.
- Unity: NullReferenceException on Play almost always means a Text object wasn't dragged into the HUD script's Inspector fields.
- A score readout stuck at 0 is almost always a missing/misplaced increment, not a display bug - confirm the underlying variable first.

A Real Scoreboard!

Score and lives, visible the whole time you play - a promise kept from Lesson 6.

Next week: game-over and restart flow - finally closing the loop Lesson 8 opened.

Try a Level 3 idea at home if you'd like - styling your HUD is a satisfying, visible one.